

TinyTorch Notebook Import Bug — Root Cause & Fix

Debugging log for: ModuleNotFoundError in mod2_activations.ipynb

Project	sandhorse-tinytorch
Environment	.venv (Python 3.1x), Windows
Symptom	ModuleNotFoundError: No module named 'tinytorch'
Root Cause	Editable install pointed at a different, unrelated project folder
Resolution	Reinstalled package in editable mode from the correct project directory
Status	Resolved — import and print(Tensor) succeed

1. The Problem

While running the first code cell of **mod2_activations.ipynb**, importing the `tinytorch` package failed, even though the package appeared in the project's file tree with a valid `core/tensor.py` module.

```
#import sys
#sys.path.append('.')
from tinytorch.core.tensor import Tensor

print(Tensor)
```

Result:

```
ModuleNotFoundError Traceback (most recent call last)
Cell In[2], line 3
1 #import sys
2 #sys.path.append('.')
----> 3 from tinytorch.core.tensor import Tensor
4
5 print(Tensor)

ModuleNotFoundError: No module named 'tinytorch'
```

The commented-out `sys.path.append('.')` lines indicated an earlier, informal workaround attempt — a sign the package had never been reliably installed as a proper Python package in this environment.

2. Diagnostic Steps

Step 1 — Confirm a build config exists

The project root contained a valid `pyproject.toml` defining the `tinytorch` package (setuptools build backend, package discovery rules, dependencies). This ruled out "no packaging config" as the cause and

pointed toward an installation/environment issue instead.

Step 2 — Identify which interpreter the notebook kernel uses

```
import sys
print(sys.executable)
```

```
c:\Users\USER\Desktop\projects\sandhorse-tinytorch\.venv\Scripts\python.exe
```

Step 3 — Check whether tinytorch is installed into that exact interpreter

```
.venv\Scripts\python.exe -m pip show tinytorch
```

Output:

```
Name: tinytorch
Version: 0.1.13
Location: C:\Users\USER\Desktop\projects\sandhorse-tinytorch\.venv\Lib\site-packages
Editable project location: C:\Users\USER\Desktop\projects\tinytorch-sandbox\tinytorch
Requires: certifi, numpy, pytest, PyYAML, rich
```

Root cause identified

The package **was** installed and pip considered it "satisfied" — but as an **editable install**, its source of truth was a path pointer (a .pth file) recorded at install time. That pointer referenced `tinytorch-sandbox`, a different project folder than the one actually being worked in (`sandhorse-tinytorch`). Since that folder had been renamed, moved, or was simply the wrong original source, Python could no longer resolve the module at import time — despite pip metadata reporting a successful install.

3. The Fix

Re-run the editable install from the correct project directory, forcing pip to overwrite the stale path pointer:

```
.venv\Scripts\python.exe -m pip install -e . --force-reinstall --no-deps
```

`--no-deps` avoided unnecessarily reinstalling `numpy`, `rich`, etc. `--force-reinstall` was required because pip otherwise treats an already-"satisfied" package as nothing to do, even when its editable pointer is broken.

Verified the pointer updated correctly:

```
Editable project location: C:\Users\USER\Desktop\projects\sandhorse-tinytorch
```

The Jupyter kernel was then **restarted** (required — a stale broken path can remain cached in `sys.path` for the life of the running kernel process) and the original cell was re-run.

Result — Success

```
from tinytorch.core.tensor import Tensor
print(Tensor)

<class 'tinytorch.core.tensor.Tensor'>
```

4. Takeaways for Future Reference

- An editable install (`pip install -e .`) stores a path pointer, not the code itself — moving, renaming, or duplicating the project folder can silently break imports even though `pip show` still reports the package as installed.
- Always check `pip show` for the "Editable project location" line when an import fails unexpectedly for a package that appears to be installed.
- Confirm the notebook kernel's interpreter (`sys.executable`) matches the environment you're installing into — mismatched kernels are a very common cause of this exact error.
- Use `--force-reinstall` when re-pointing an editable install; pip will otherwise skip an already-"satisfied" package even if its source location is stale or invalid.
- Always restart the kernel after reinstalling a package — a running Python process caches import state and won't pick up the fix otherwise.

Documented for the sandhorse-tinytorch project — reference this if the "ModuleNotFoundError: No module named 'tinytorch'" issue recurs after moving/cloning the repository.